

Scheduling 200 students against 15 supervisors and 5 labs

A department runs office hours and lab slots on shared spreadsheets. It fails the same four ways every week — and every one of them is an availability-engine problem.

01 Two students, one slot

Nothing makes a booking atomic. The original checks availability, then writes seconds later, with no transaction in between.

02 Wasted capacity

Most of the week sits empty while a few hours are oversubscribed. Nothing measures demand against supply.

03 Nobody trusts the calendar

A booking that "succeeded" isn't guaranteed to be reserved, so staff and students keep private copies of the truth.

04 Time zones and term calendars ignored

Visiting faculty and exchange students get the wrong hour; holidays and reading weeks are treated as working days.

200

students booking

~15

faculty office hours

5

labs, with stations

3

Killer Tests, live

Slots as range algebra, not guesswork

The engine computes `working hours - busy - buffers - overrides - holidays` as set subtraction over date ranges. Each stage is a pure function, so each is testable alone.

1-3

Resolve hours

recurring rules → date overrides
replace a day → holidays subtract it

4-5

Load & inflate busy

bookings + live holds + the
student's own timetable, then grow
by buffers

6

Subtract

range algebra in both directions —
never a filter on start times

7-8

Slice & gate

duration + spacing, then drop
anything inside the notice window

9-10

Project & rank

into the booker's zone, then rank.
Ranking never removes a legal
slot.

slots,
in the
booker's
zone

KILLER TEST 1 · TIME ZONES

Stage 9. One zone module, driven by the zone
database. No offset constants exist anywhere in the
codebase.

KILLER TEST 2 · BUFFERS

Stage 5. Busy intervals are **inflated**, taking the max of
the adjacent buffers — so the gap is correct in both
directions.

KILLER TEST 3 · ONE WINNER

The booking path. A single-writer module holds the
transaction-scoped lock. Nothing else may insert a
booking.

The obvious fix is broken. We have the **measurement**.



WHY THE GUARD FAILS — AND WHY A TEST WOULD NEVER TELL YOU

Under `READ COMMITTED` the `NOT EXISTS` subquery is a snapshot read: every concurrent transaction sees zero existing rows and proceeds. Only a *committed* competitor is visible.

And it hides well — with an exclusion constraint added, capacity-1 slots look **perfect** while multi-station lab slots silently admit three times their capacity. **A double-booking bug you only find by testing capacity.**

THE CORRECTED CLAIM

```
BEGIN;  
SELECT pg_advisory_xact_lock(  
    hashtextextended(host || ':' || day, 0));  
-- count overlapping bookings + live holds  
-- if >= resources_needed -> ROLLBACK, 409  
INSERT INTO bookings (...) RETURNING id;  
COMMIT;
```

Meridian Intelligence

Not a feature the original has — four it structurally cannot have. Verified by search, not assumed: docs/OBSERVATIONS.md §F.

D3

Clash Sentinel — the strongest one

A student with a 14:00 lecture is **never offered** a slot that collides with it. A rejecting response names the specific conflict and offers the nearest alternatives.

The original reads the *host's* calendar and never the invitee's (getUserAvailability.ts:648). Modelling the booker's own timetable is impossible in its architecture — not merely absent.

D1

Concierge

"45 min with Prof. Rao before Friday, I'm busy 2–4 Wednesday" → a genuinely bookable slot.

D2

Allocator

Distributes scarce slots across a cohort by a stated fairness policy; reports demand, supply, served, unfilled.

D4

Suggest

Ranks slots from what has worked before. Ranking only.

IT CANNOT INVENT AVAILABILITY

The model returns **constraints** — duration, host, windows — never slots. Every candidate is validated by the same engine that serves the public slot list.

IT RUNS WITH THE AI OFF

Empty API key → identical results from a structured form. D2–D4 need no network at all. This is a **tested** configuration, not a promise.

IT NEVER REPLACES THE HARD CORE

Switch it all off and Killer Tests 1, 2 and 3 still pass unchanged. **Verified, because the Brief requires it.**

What we measured, and what we didn't

CLAIM	HOW IT IS CHECKED	VERIFIED
Exactly one booking wins a contested slot	40 concurrent callers from a common barrier, × (host , day) lock, constraint disabled	1 winner ×5 runs
A 3-station lab admits exactly 3	8 concurrent callers, capacity 3 — the case a blanket constraint breaks	3 admitted
The harness can actually detect the defect	negative control: unguarded insert, 20 callers, one slot	20 rows
IST host and PST booker see identical instants	zone-database conversion, asserted on instant identity, not clock strings	12h30m apart
Every claim about the original is openable	202 citation lines resolved — file exists, line in range, non-blank, says it	202 / 202
The app runs with no AI key	start with OPENAI_API_KEY empty; browse, book, cancel	all pass

ONE HONEST LIMITATION

The original was **not executed**. The concurrency defect is proved from its code structure and from three open issues on its own tracker (#29605, #29958, #29967), and reproduced independently against our schema — not observed live in the original. We say so rather than implying we ran it.

WHAT WE DELIBERATELY REJECTED

Redis lock — puts the network in the correctness path. **Blanket EXCLUDE** — would break lab capacity. **Calendar sync** — an integration problem, not an engine one, and not in the Brief.